Scientific Interest in AMR

Scientific Interest in AMR

Overview

The core goal is to define **scientific interest** for a pathogen-antibiotic pair as the amount of AMR publication attention that remains after accounting for broad background attention to the pathogen and the antibiotic.

The starting quantities are:

- `count`: the observed AMR-positive publication count for one `(pathogen, antibiotic, interval)` row
- `bg_pathogen_n`: the total publication background for the pathogen in the same interval
- `bg_antibiotic_n`: the total publication background for the antibiotic in the same interval

These background marginals do not encode resistance by themselves. Instead, they define a reasonable **background-attention baseline**. The model then asks:

> Given only the general publication attention to this pathogen and this antibiotic, what AMR count would be typical?

Pooled baseline model

The pooled model is fit on the log scale:

```
log1p(count) = alpha + u_pathogen + v_antibiotic + beta_path *
log1p(bg_pathogen_n) + beta_ab * log1p(bg_antibiotic_n)
```

This is a global pooled model with pathogen and antibiotic effects.

Meaning of the terms

Using ***Escherichia coli* resistance to ciprofloxacin** as a running example:

- `count` is the observed number of AMR-positive papers for that exact pair in one interval

- `log1p(count)` is the response on the log scale, using `log(1 + x)` so that zero counts are handled naturally and large counts are compressed
- `alpha` is the global intercept, representing the baseline AMR publication level before pair-specific or background-specific information is added
- `u_pathogen` is the pathogen-specific pooled effect For *E. coli*, this captures whether *E. coli* tends to attract more or fewer AMR publications than average across all antibiotics
- `v_antibiotic` is the antibiotic-specific pooled effect For ciprofloxacin, this captures whether ciprofloxacin tends to attract more or fewer AMR publications than average across all pathogens
- `bg_pathogen_n` is the total publication background for the pathogen in that interval, regardless of whether resistance is discussed
- `bg_antibiotic_n` is the total publication background for the antibiotic in that interval, again regardless of whether resistance is discussed
- `beta_path * log1p(bg_pathogen_n)` is the contribution of general pathogen attention
- `beta_ab * log1p(bg_antibiotic_n)` is the contribution of general antibiotic attention

So a high expected value for *E. coli* and ciprofloxacin can arise because:

- *E. coli* is broadly important in AMR research
- ciprofloxacin is broadly important in AMR research
- one or both receive substantial general publication attention in that interval

The model combines all of these sources into one pooled baseline expectation.

How the parameters are computed

All parameters are estimated jointly from the full interval-level dataset, not from one pair at a time.

For each row, the model uses:

- response: `log1p(count)`
- continuous predictors: `log1p(bg_pathogen_n)` and `log1p(bg_antibiotic_n)`
- one-hot indicator columns for pathogen identity
- one-hot indicator columns for antibiotic identity

So if one row corresponds to *E. coli* and ciprofloxacin, the design vector contains:

- the two log-background values for that interval
- a `1` in the *E. coli* pathogen column
- a `1` in the ciprofloxacin antibiotic column
- `0` in the other pathogen and antibiotic columns

The model is then fit across all rows at once using **ridge regression** on the log scale.

In practice:

- `alpha` is the fitted intercept
- `beta_path` is the fitted coefficient on `log1p(bg_pathogen_n)`
- `beta_ab` is the fitted coefficient on `log1p(bg_antibiotic_n)`
- each `u_pathogen` value is the coefficient of that pathogen's one-hot indicator
- each `v_antibiotic` value is the coefficient of that antibiotic's one-hot indicator

The ridge penalty shrinks coefficients toward zero. This stabilizes the fit when many pathogens and antibiotics are represented and when some pairs are sparse.

In code, the design matrix contains:

- two numeric columns: `log_bg_pathogen_n` and `log_bg_antibiotic_n`
- one-hot columns for all pathogens
- one-hot columns for all antibiotics

The fitted model predicts:

```
expected_log_count_global = alpha + u_pathogen + v_antibiotic +
beta_path * log1p(bg_pathogen_n) + beta_ab * log1p(bg_antibiotic_n)
```

For the running example, the expected log-count for *E. coli* and ciprofloxacin in a given interval is computed by plugging in:

- the fitted global intercept
- the fitted *E. coli* coefficient
- the fitted ciprofloxacin coefficient
- the observed `bg_pathogen_n` for *E. coli* in that interval
- the observed `bg_antibiotic_n` for ciprofloxacin in that interval

This yields the pooled baseline for what AMR publication volume would be expected before checking whether the pair is unusually over- or under-studied.

Scientific Interest definition

Once the pooled baseline is fitted, scientific interest is defined as:

```
SI = log1p(count) - expected_log_count_global
```

Under this definition:

- positive `SI` means more AMR publications than expected under the pooled background baseline
- negative `SI` means fewer AMR publications than expected

This is not a claim that the marginals explain resistance. They do not. Instead, the marginals define a null publication baseline, and `SI` is the residual or surprise relative to that baseline.

Why counts are fit before SI

For smoothing and extrapolation, the workflow does not fit `SI` directly. Instead, it first fits the count trajectory and then derives `SI` from that fitted count.

This keeps the modeling aligned with the actual question:

- first estimate a smooth trajectory for AMR publication counts
- then measure scientific interest as the gap between that trajectory and the pooled background expectation

That makes the model easier to inspect, because the count fit and the SI curve have a direct relationship.

LOESS count model

The simplified approach is:

1. keep the pooled global baseline `expected_log_count_global`
2. fit a LOESS smoother directly to the observed pair trajectory `log1p(count)` over time
3. use that fitted log-count trajectory as the smooth estimate of future AMR count
4. derive fitted or predicted `SI` as:

```
predicted_SI = predicted_log_count_loess - expected_log_count_baseline
```

So the red SI curve is the SI implied by the LOESS fit on log-counts.

Why LOESS was chosen

The LOESS model is intentionally modest. It does not try to separate:

- long-term trend
- residual Gaussian processes
- future background models
- multiple interacting uncertainty layers

It asks only:

> What smooth log-count trajectory best matches this pair in the chosen fit window?

Once that trajectory is estimated, comparing it with the pooled background expectation gives the fitted `SI`.

Extrapolation beyond the fit window

After fitting LOESS inside the training window, the model must decide how to continue the count trajectory beyond the last fitted year.

The extrapolation is done on the **log-count scale**. That matters because:

- a straight line in `log1p(count)` corresponds to multiplicative growth or decline on the count scale
- even a moderate positive slope on the log scale can become a strong increase when transformed back to counts

The current implementation extrapolates from the **boundary behavior of the fitted curve**, not from a fresh model fit outside the training range.

Boundary slope

At the right edge of the fit window, the model computes a boundary slope from the fitted LOESS curve itself. This is more stable than using the raw last two observed points, which can be overly sensitive to local noise.

The forecast then starts from:

- the fitted log-count value at the boundary
- the fitted LOESS slope at that boundary

Optional dampening

The extrapolated tail can be either undamped or damped.

- `tail_damping_years=None` means **no damping** The trajectory continues with a linear tail on the log-count scale, so if the boundary slope is positive, the forecast keeps rising indefinitely.
- `tail_damping_years=<float>` means **damped extrapolation** The boundary slope is preserved initially, but it gradually decays toward zero over the specified dampening timescale.
- omitting `tail_damping_years` lets the code use its internal automatic heuristic

So dampening controls how quickly the extrapolated trajectory flattens after leaving the fit window.

Conceptually:

- no damping: keep the boundary slope forever
- damping: keep the initial direction, but reduce the slope progressively over future years

This is useful because some pairs show a real upward trend near the end of the fit window, but a fully undamped log-linear forecast can become unrealistically aggressive over long horizons.

Uncertainty interpretation

The `±1 std` uncertainty band is based on the residual spread of the training log-counts around the LOESS fit.

This is:

- an empirical spread band
- not a formal predictive interval

Plot interpretation

In the interval-level LOESS plots:

- orange line: LOESS fit of `log1p(count)`, converted back to counts
- red line: `SI` implied by that LOESS count fit relative to the pooled baseline
- navy line: observed `SI`
- bars: observed per-interval AMR count and the two marginals on the secondary axis

This preserves a simple interpretation: smooth the counts first, then express scientific interest from those smoothed counts.

Yearly data

In the current workflow, the observed rows are true annual AMR counts built from PMID-level publication dates.

The yearly modeling pipeline is:

- construct annual `(pathogen, antibiotic, year)` AMR counts
- merge annual pathogen and antibiotic background counts
- fit the pooled expected-count baseline
- define `SI` as the residual from that baseline
- fit the count trajectory rather than fitting `SI` directly
- derive predicted `SI` from the predicted count trajectory

What to inspect after fitting

After fitting the pooled model, the most informative quantities to inspect are:

- `expected_count_global`: the background-only expected AMR count
- `SI`: the residual or surprise score on the log scale
- the coefficients on `log_bg_pathogen_n` and `log_bg_antibiotic_n`

There are many one-hot pathogen and antibiotic effects, so they are usually not printed in full by default. The two continuous coefficients are often the clearest summary of how expected AMR volume scales with general pathogen and antibiotic attention.